CroustyQuizz

Rendu 1 · Dossier de lancement · M1 MIAGE · Université Paris Nanterre
Année 2026–2027 · Révision du 10 novembre 2026

Équipe, comptes développeurs et fonctionnalités majeures

Notre équipe réunit **trois étudiants**. Chacun prend en charge une fonctionnalité métier majeure, de sa conception à ses tests, avec ses échanges API, sa persistance et ses interfaces.

Étudiant	Compte GitHub personnel	Fonctionnalité majeure et validation prévue
Thibaud Thomas-Lamotte	Tibxla	**Karaoké** : File d'attente, Passages, paroles synchronisées, Vote du public et Palmarès du Karaoké. Démontrer un Passage complet et l'interdiction de voter pour soi.
Harris Hamadache	Piyakabib	**Quiz** : préparation des Questions, Réponses horodatées, correction et points selon la rapidité. Démontrer une Manche complète et le refus d'une seconde Réponse différente.
Rayen Ndiaye	nRayen	**Blind test** : sélection des Morceaux, ordre des Buzz, propositions de titre et d'artiste, correction et points. Démontrer la prise de main et le blocage après une erreur.

Les contributions transverses suivent la répartition du PRD : Thibaud porte le contrat commun de Manche, le temps réel et le Pilotage Automatique ; Harris, les comptes Animateur, la Console, le Classement général, la modération et les Contacts du Lieu ; Rayen, les Soirées, la reconnexion, le Pilotage Manuel, le Format Équipe et le serveur MCP. Les commits et PR utilisent les comptes individuels ci-dessus pour rendre les contributions évaluables.

Projet et caractère innovant

Un **Lieu** accueillant du public doit pouvoir préparer et animer une Soirée sans disperser les réponses, les sélections musicales et les résultats entre plusieurs outils. CroustyQuizz vise une plateforme web commune à trois Modes de jeu : Quiz, Blind test et Karaoké. L'Animateur pilote depuis la **Console d'animation**, les Joueurs utilisent leur téléphone comme **Manette**, et le public suit l'**Écran de salle**.

L'innovation d'usage proposée repose sur la continuité de la Soirée : **un seul accès par QR code**, une Manette qui s'adapte à chaque Manche et un Pilotage Automatique avec intervention possible de l'Animateur. La préparation conversationnelle par un Assistant externe complète cette approche. L'objectif est de réduire les manipulations pendant l'animation ; cette valeur devra être vérifiée lors de démonstrations.

Dépôt principal et suivi

Dépôt unique, front, API et back : https://github.com/Tibxla/croustyquizz
Spécification de référence : PRD, issue nº 2.

Version de ce rendu : doc/rendu-1 sur `develop`, fusionnée par la PR nº 36.

Le dossier est maintenu dans **`doc/rendu-1/dossier-lancement.md`**, compilé en **`doc/rendu-1/dossier-lancement.pdf`** et destiné à être versionné avec les livraisons. Les issues, branches par fonctionnalité et PR relues retracent le travail ; les conventions figurent dans `CONTRIBUTING.md`.

Scénarios d’usage prévus

Les parcours suivants décrivent la **cible fonctionnelle**. Le premier parcours à réaliser de bout en bout est une Soirée en Format Solo et Pilotage Manuel avec une Manche de Quiz.

1. Préparer et ouvrir une Soirée

Acteur : l’Animateur. Après connexion à la Console d’animation, il crée une Soirée, choisit le Format Solo ou Équipe et le Pilotage Manuel ou Automatique. Il ajoute et ordonne des Manches, sélectionne leurs contenus et règle les durées. Il ouvre les inscriptions puis affiche l’Écran de salle. **Résultat :** le programme est prêt et le QR code permet de rejoindre la Soirée.

2. Rejoindre, jouer au Quiz et se reconnecter

Acteurs : l’Animateur et les Joueurs. Un Joueur scanne le QR code ou saisit le code d’accès, choisit un pseudo et accède à sa Manette sans compte ni installation. L’Animateur lance la Manche de Quiz. Pour chaque Question, le Joueur choisit une des quatre propositions avant la fin du délai ; le serveur calcule ses points selon la justesse et la rapidité. La correction puis le Classement général sont affichés. **Cas prévu :** après une coupure, le Joueur retrouve ses points et sa Réponse enregistrée, sans répondre à nouveau ni rejouer les Questions manquées.

3. Prendre la main au Blind test

Acteurs : les Joueurs et l’Animateur. L’Écran de salle diffuse un extrait du Morceau. Les Joueurs buzzent ; le serveur ordonne les Buzz reçus et donne la main au premier Concurrent. Celui-ci propose le titre ou l’artiste sur sa Manette ; la correction tolère les petites fautes. Une erreur le bloque jusqu’au Morceau suivant ; en Pilotage Manuel, l’Animateur peut arbitrer le verdict. **Résultat :** les points du titre et de l’artiste alimentent le Classement général.

4. Chanter et voter au Karaoké

Acteurs : le Joueur appelé, le public et l’Animateur. Un Joueur choisit une Chanson et s’inscrit dans la File d’attente, avec un seul Passage en attente à la fois. Lorsqu’il est appelé, il confirme qu’il est prêt ; la musique et les paroles synchronisées accompagnent son Passage. Le public vote ensuite de 1 à 5, sans pouvoir voter pour son propre Passage. En Pilotage Automatique, une absence de confirmation sous 30 secondes fait passer au suivant. **Résultat :** le Palmarès du Karaoké reste séparé du Classement général.

5. Jouer en Équipe et gérer un imprévu

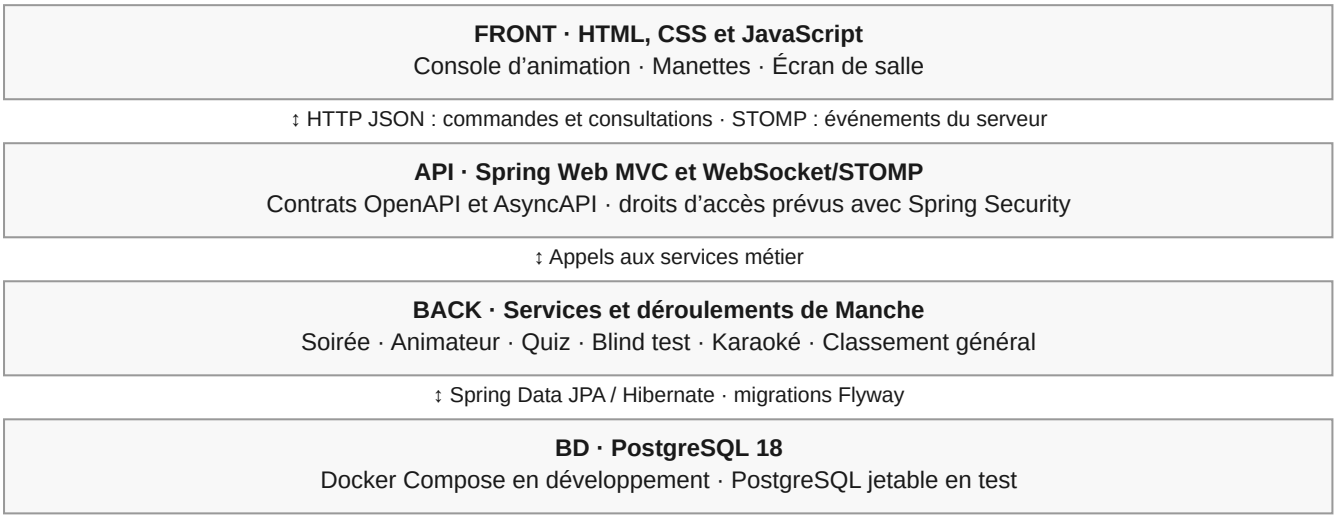
En Format Équipe, les Joueurs créent ou rejoignent une Équipe sans capitaine. Sa première Réponse ou son premier Buzz l’engage au Quiz et au Blind test ; le Karaoké reste individuel. Quel que soit le Pilotage, l’Animateur peut mettre en pause, reprendre ou passer une étape. Il traite les Signalements et peut demander un changement de pseudo par Avertissement ou décider une Exclusion.

6. Préparer avec un Assistant et prolonger la relation

L’Animateur autorise son Assistant externe avec une Clé d’Assistant propre au Lieu. Celui-ci prépare un programme et propose des Questions en français ; l’Animateur les relit dans la Console. Les Soirées En cours ou Terminées ne sont pas modifiables par l’Assistant. En complément, un Joueur peut devenir Contact du Lieu avec un Consentement facultatif et révocable pour les Campagnes SMS. Ce complément reste secondaire par rapport aux trois Modes de jeu.

Architecture et frameworks envisagés

L'architecture retenue est un **monolithe modulaire Spring Boot 4.1.1**, en **Java 21**, construit avec le wrapper **Maven**. Une seule application sert les pages et l'API ; les packages métier sous `fr.croustyquizz` permettent de répartir le développement.



Architecture cible : le serveur contrôle les règles et diffuse un état adapté à chaque écran.

Front et contrats réseau

Les trois interfaces sont des pages HTML et JavaScript servies directement par Spring Boot, **sans framework front imposé ni bundler** (ADR 0001). Une bibliothèque peut être chargée par script. Les contrats **OpenAPI pour HTTP** et **AsyncAPI pour STOMP** du premier parcours de Quiz sont écrits dans `docs/api/` (issue nº 34) et validés par la CI avant toute implémentation (ADR 0003). Réponses, Buzz et Votes passent par HTTP ; STOMP diffuse les événements vers la Console, la salle ou une Manette selon leurs droits.

Le serveur décide de l'identité, des délais, de l'ordre des actions et des points. Répéter le même envoi de Réponse doit restituer son résultat initial sans nouvel effet ; modifier une Réponse acceptée est refusé. La reconnexion récupère une vue autorisée de l'état courant, sans dévoiler une bonne réponse avant la correction.

Back et persistance

Chaque Mode de jeu implémente `DeroulementDeManche`. Une `Horloge` et un `DiffuseurTempsReel` injectés permettent de tester les délais et les événements sans attendre le temps réel. Spring Data JPA et Hibernate accèdent à PostgreSQL ; **Flyway** versionne le schéma. Un Lieu organise des Soirées, qui contiennent des Manches, des Joueurs et éventuellement des Équipes. Les données propres aux Questions, Buzz, Passages et Votes complètent ce socle ; les diagrammes sont dans `docs/conception/`.

Intégrations et décisions encore ouvertes

Le catalogue Deezer est envisagé pour les extraits du Blind test et LRCLIB pour les paroles synchronisées. **La source audio du Karaoké reste à valider** : les paroles ne fournissent pas la musique. La banque française de Questions et le prestataire SMS restent également à choisir. Des ports métier isolent ces dépendances. Le serveur MCP envisagé avec Spring AI adapte les mêmes services pour l'Assistant externe (ADR 0002) ; CroustyQuizz n'appelle aucun modèle de langage.

Protocole d’IA, livraisons et qualité

Protocole d’usage de l’IA pour le développement

Nous utilisons l’IA comme aide à la conception, au développement, aux tests et à la rédaction de la documentation. Nous lui fournissons le contexte du projet et une tâche précise, puis nous relisons ses propositions avant de les intégrer. Les choix techniques et la validation des livrables restent à la charge de l’équipe.

1. **Tâches et outils.** Nous sollicitons notamment Claude Code, Codex et Antigravity CLI pour comprendre du code, comparer des pistes de conception, proposer des implémentations, rechercher des cas de test et améliorer la documentation. Les demandes s’appuient sur le glossaire, les décisions d’architecture et les conventions du dépôt.
2. **Traçabilité.** Les issues, les commits et les PR relues retracent chaque contribution et son auteur. Nous conservons les échanges utiles avec l’IA pour expliquer nos décisions.
3. **Maîtrise et relecture.** L’auteur comprend et sait expliquer toute contribution intégrée. Il vérifie les propositions dans le code et les sources techniques, corrige les erreurs et obtient la relecture d’un autre membre.
4. **Vérification.** Les tests pertinents et la CI sont exécutés avant fusion. Nous contrôlons les règles métier, les délais, les droits d’accès et les dépendances ; une affirmation de l’IA ne prouve pas la réussite d’un test.
5. **Données transmises.** Nous utilisons des exemples fictifs et ne transmettons ni secrets d’accès ni données personnelles réelles. Les clés et mots de passe restent hors du dépôt et des échanges avec l’IA.

L’**Assistant de l’Animateur** est une fonctionnalité distincte de ce protocole de développement : ses accès seront limités au Lieu, sa clé révocable et ses Questions relues avant utilisation.

Livraisons, validation et état du projet

Les jalons prévus sont **`v0.1.0` pour le socle**, **`v0.2.0` pour les Modes de jeu**, puis **`v1.0.0` pour le proof-of-concept final**. Chaque livraison fera évoluer ce dossier et présentera les fonctionnalités, tests et limites. `develop` reçoit les PR relues avec CI verte ; les livraisons passent ensuite sur `main` avec un tag, selon `CONTRIBUTING.md`.

Le socle comprend le démarrage Spring Boot, la page d’accueil, PostgreSQL/Flyway, le contrat de Manche et ses tests sur un mode factice. Les Modes de jeu, tables métier et authentification restent à développer ; les contrats réseau du premier parcours sont écrits et le temps réel est en cours. **La CI** lance `./mvnw verify`, mesure la couverture et analyse le code ; **la livraison** publie une image Docker sur GHCR et, à chaque tag, une release avec l’application, que la démo installe à la demande.

Les tests prévus couvrent les règles des trois modes, les délais avec une Horloge factice, les migrations et les parcours HTTP/STOMP sur PostgreSQL avec **Testcontainers**. Les objectifs de validation incluent la reconnexion, les envois répétés et une salle de **60 Joueurs simultanés** ; ce sont des objectifs, pas des résultats déjà obtenus.

README, licence et indicateurs

Le README présente l’installation et les badges de **version**, **licence MIT**, **couverture JaCoCo** et **alertes PMD**. `./mvnw verify` produit les rapports ; PMD bloque le build en cas d’alerte. La CI conserve les rapports et publie les badges du dernier build réussi sur la branche `mesures-qualite`. Ces indicateurs portent sur le code existant. Le socle est en `0.0.1-SNAPSHOT` ; les tags ci-dessus sont prévus et la licence figure dans `LICENSE`.

Références : consignes du Rendu 1 ; fiche officielle de l’EC Projet, consultée le 10/11/2026 ; PRD nº 2 ; GLOSSARY.md ; CONTRIBUTING.md ; pom.xml ; workflow CI ; ADR 0001 à 0003 ; issue nº 34 et `docs/api/`. Les décisions ouvertes seront révisées aux prochaines livraisons.